

SIMATS ENGINEERING

CO3-SCENARIO BASED ASSIGNMENT

COURSE CODE: CSA 4301

COURSE NAME : internet programming

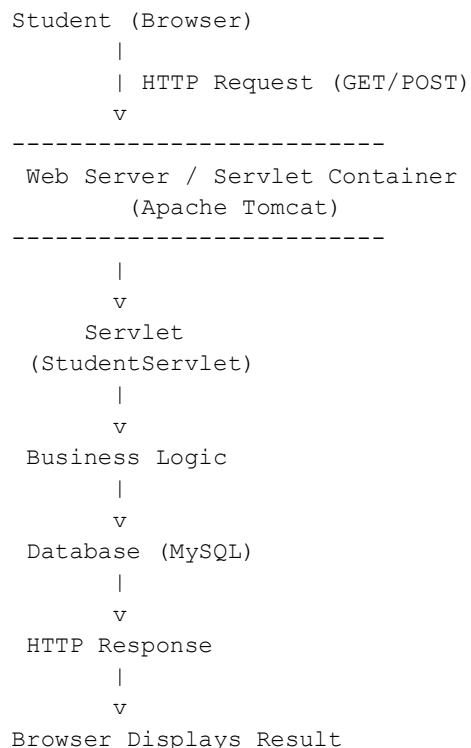
NAME: Ganesh Reddy

Reg.no: 192311153

Question 1: Servlet-Based Student Registration System

(a) Architecture of a Servlet-Based Application

A servlet-based web application follows a client-server architecture. The student uses a web browser to submit an HTTP request. The web server or servlet container, such as Apache Tomcat, receives the request and invokes the appropriate servlet. The servlet processes the request, performs business logic or database operations, and sends an HTTP response back to the browser.



Components

- Client (Browser): Sends HTTP requests.
- Web Server/Servlet Container: Receives requests and manages servlet execution.
- Servlet: Processes requests and generates responses.
- Database: Stores student information.
- Response: HTML/JSP content sent back to the client.

(b) Student Registration Servlet

HTML Form

```
<!DOCTYPE html>
<html>
<head>
    <title>Student Registration</title>
</head>
<body>
<form action="StudentServlet" method="post">
    Name: <input type="text" name="name"><br><br>
    Email: <input type="email" name="email"><br><br>
    Course: <input type="text" name="course"><br><br>
    <input type="submit" value="Register">
</form>
</body>
</html>
```

StudentServlet.java

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class StudentServlet extends HttpServlet {

    protected void doGet(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        String name = request.getParameter("name");
        String email = request.getParameter("email");
        String course = request.getParameter("course");

        out.println("<h2>Student Details (GET)</h2>");
        out.println("Name: " + name + "<br>");
        out.println("Email: " + email + "<br>");
        out.println("Course: " + course);
    }

    protected void doPost(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        String name = request.getParameter("name");
        String email = request.getParameter("email");
        String course = request.getParameter("course");

        out.println("<h2>Registration Successful</h2>");
        out.println("Name: " + name + "<br>");
        out.println("Email: " + email + "<br>");
        out.println("Course: " + course);
    }
}
```

```
}  
}
```

(c) Servlet Life Cycle

The servlet life cycle consists of three main stages: `init()`, `service()`, and `destroy()`.

1. `init()`

The `init()` method is called only once when the servlet is initialized. It is used to initialize resources.

```
public void init() {  
    System.out.println("Servlet Initialized");  
}
```

2. `service()`

The `service()` method is called for every client request. It determines the HTTP method and internally invokes `doGet()`, `doPost()`, or another appropriate method.

```
Client Request  
    |  
    service()  
    /      \  
doGet()   doPost()
```

3. `destroy()`

The `destroy()` method is called before the servlet is removed from service. It is used to release resources.

```
public void destroy() {  
    System.out.println("Servlet Destroyed");  
}
```

(d) GET vs POST

Feature	GET	POST
Data location	URL/query string	Request body
Data exposure	Visible in URL	Not normally visible in URL
Data size	More limited	Supports larger data
Bookmarking	Can be bookmarked	Not normally bookmarked
Typical use	Fetching data	Submitting data

POST is more appropriate for handling sensitive form data because the submitted values are carried in the request body rather than being exposed in the URL. However, POST alone does not encrypt data. Sensitive information should always be transmitted over HTTPS.

Question 2: Session Management Using HttpSession and Cookies

(a) Session Tracking

HTTP is a stateless protocol, meaning that each request is independent. Session tracking allows a web application to identify a user and maintain information across multiple requests and pages.

Common session tracking techniques include HttpSession, Cookies, URL Rewriting, and Hidden Form Fields.

(b) Login Servlet Using HttpSession

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class LoginServlet extends HttpServlet {

    protected void doPost(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {

        String user = request.getParameter("username");
        String pass = request.getParameter("password");

        if (user.equals("admin") && pass.equals("123")) {

            HttpSession session = request.getSession();
            session.setAttribute("username", user);

            response.sendRedirect("HomeServlet");

        } else {
            response.getWriter().println("Invalid Login");
        }
    }
}
```

Home Servlet

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class HomeServlet extends HttpServlet {

    protected void doGet(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {

        HttpSession session = request.getSession(false);

        if (session != null) {
            String user = (String) session.getAttribute("username");
            response.getWriter().println("Welcome " + user);
        } else {
            response.getWriter().println("Please Login");
        }
    }
}
```

(c) Using Cookies

```
Cookie cookie = new Cookie("username", user);
cookie.setMaxAge(3600);
response.addCookie(cookie);
```

Reading a Cookie

```
Cookie cookies[] = request.getCookies();
```

```

if (cookies != null) {
    for (Cookie c : cookies) {
        if (c.getName().equals("username")) {
            out.println(c.getValue());
        }
    }
}
}

```

(d) Session Tracking Flow

```

Login Page
  |
  v
Login Servlet
  |
Creates HttpSession
  |
Stores username
  |
Browser receives session ID cookie
  |
Subsequent Requests
  |
Session Retrieved
  |
User Remains Logged In

```

(e) HttpSession vs Cookies

Feature	HttpSession	Cookies
Storage	Primarily server-side	Client-side
Security	Session data stays on server	Cookie values can be accessed by client
Data type	Can store Java objects	Stores text values
Purpose	Login/session state	Preferences and identifiers
Lifetime	Controlled by session timeout/logout	Can be session or persistent

HttpSession is generally preferred for authenticated user state because application data can be maintained on the server. Cookies are commonly used to carry the session identifier or store non-sensitive preferences.

Question 3: JDBC-Based E-Commerce Application

(a) JDBC Architecture

```

Java Application
  |
  v
JDBC API
  |
  v
JDBC DriverManager
  |
  v
JDBC Driver
  |

```

Components

- Java Application: Contains the application logic.
- JDBC API: Provides Java interfaces and classes for database access.
- DriverManager: Manages JDBC drivers and helps establish connections.
- JDBC Driver: Converts JDBC calls into database-specific operations.
- Database: Stores product information.

(b) Steps in JDBC Connectivity

1. Import the JDBC package.
2. Load/register the JDBC driver.
3. Establish a database connection.
4. Create a Statement or PreparedStatement.
5. Execute the SQL query.
6. Process the ResultSet for SELECT queries.
7. Close ResultSet, Statement, and Connection.
8. Handle exceptions properly.

(c) Insert Product Data

```
import java.sql.*;

public class InsertProduct {

    public static void main(String args[]) {

        try {
            Class.forName("com.mysql.cj.jdbc.Driver");

            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/shop",
                "root",
                "password");

            String sql = "INSERT INTO product VALUES(?,?,?)";

            PreparedStatement ps = con.prepareStatement(sql);

            ps.setInt(1, 101);
            ps.setString(2, "Laptop");
            ps.setDouble(3, 65000);

            int rows = ps.executeUpdate();

            System.out.println(rows + " Record Inserted");

            ps.close();
```

```

        con.close();

    } catch (ClassNotFoundException e) {
        System.out.println("JDBC Driver not found");
    } catch (SQLException e) {
        System.out.println("Database Error: " + e.getMessage());
    }
}
}

```

(d) Retrieve Product Data

```

import java.sql.*;

public class DisplayProduct {

    public static void main(String args[]) {

        try {
            Class.forName("com.mysql.cj.jdbc.Driver");

            Connection con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/shop",
                "root",
                "password");

            Statement st = con.createStatement();

            ResultSet rs =
                st.executeQuery("SELECT * FROM product");

            while (rs.next()) {
                System.out.println(
                    rs.getInt(1) + " " +
                    rs.getString(2) + " " +
                    rs.getDouble(3));
            }

            rs.close();
            st.close();
            con.close();

        } catch (ClassNotFoundException e) {
            System.out.println("JDBC Driver not found");
        } catch (SQLException e) {
            System.out.println("Database Error: " + e.getMessage());
        }
    }
}

```

(e) Exception Handling and Best Practices

```

try {
    // Database operations
} catch (ClassNotFoundException e) {
    System.out.println("Driver not found");
} catch (SQLException e) {
    System.out.println("Database Error");
} finally {

```

```
    // Close database resources  
}
```

- Use PreparedStatement for parameterized SQL and to reduce SQL injection risk.
- Close ResultSet, Statement/PreparedStatement, and Connection after use.
- Use try-with-resources where possible for automatic resource management.
- Handle SQLException separately so database errors can be diagnosed clearly.
- Do not hard-code real production passwords; use secure configuration.

Overall Summary

Question 1 explains servlet architecture, GET/POST request handling, the servlet life cycle, and why POST over HTTPS is appropriate for sensitive form submissions. Question 2 explains session tracking using HttpSession and Cookies and compares server-side sessions with client-side cookies. Question 3 explains JDBC architecture and demonstrates database connectivity, product insertion, retrieval, and exception handling.